

Eight Experiments with a Self-Applicable Dependently Typed Normaliser

Executable examples from the C++23 core

16 August 2026

Abstract

This report uses the implemented normaliser to solve four elementary and four medium problems. The examples are deliberately small enough to fit the current core: de Bruijn syntax, predicative universes, dependent Pi types, quotation, unquotation, closures, neutral reflection, and eta-long reification. Each result below was obtained by compiling and running `normaliser.cpp`; the output is recorded verbatim. The medium examples use higher-order composition, dependent identity, nested staging, and open dependent application.

1 Method

For each term t in context Γ , the executable performs

$$\Gamma \vdash t : A, \quad \text{nf}_\Gamma(t) = \text{reify}_A(\text{eval}(\text{reflect}(\Gamma), t)).$$

The checker rejects ill-typed applications before evaluation. De Bruijn substitution instantiates dependent codomains, while NbE evaluates closures and reifies them into eta-long syntax. Quotation is inert; unquotation crosses the explicit stage boundary.

We use $A = \text{Type}_0$, and write x_i for de Bruijn variable i . The executable's complete command is:

```
g++ -std=c++23 -Wall -Wextra -pedantic -O2 normaliser.cpp -o normaliser
./normaliser
```

2 Four elementary problems

2.1 Problem 1: identity application

$$(\lambda : A.x_0)x_0$$

in context $x_0 : A$. Beta reduction substitutes the argument for the bound variable. The normaliser returns

$$\boxed{x_0}.$$

2.2 Problem 2: nested administrative redexes

$$(\lambda : A.x_0)((\lambda : A.x_0)x_0).$$

Two beta contractions are performed. The result is again x_0 . This checks that normalization recursively reduces an argument before applying a closure.

2.3 Problem 3: quotation round trip

$$\text{unquote}(()\text{quote}((\lambda : A.x_0)x_0)).$$

Quotation stores syntax without executing it; unquotation restores and executes the stored term. The result is

$$\boxed{x_0}.$$

The type checker assigns the quoted expression $\text{Code}(A)$, so the round trip is typed rather than an untyped parser trick.

2.4 Problem 4: eta-long neutral function

Let $F = (A) \rightarrow A$ and consider the open neutral $x_0 : F$. Since the normal form convention is eta-long, the normaliser expands it to

$$\boxed{\lambda : A.x_1 x_0}.$$

The shift from x_0 to x_1 is the de Bruijn adjustment required when the fresh lambda binder is introduced.

3 Four medium problems

3.1 Problem 5: typed higher-order composition

Define the typed composition term

$$\text{compose} : (A \rightarrow A) \rightarrow (A \rightarrow A) \rightarrow A \rightarrow A$$

by

$$\lambda f.\lambda g.\lambda x.f(gx).$$

Applying it to two identity functions and $x_0 : A$ gives

$$\text{compose } (\lambda : A.x_0) (\lambda : A.x_0) x_0.$$

The normaliser returns x_0 . This combines three dependent function applications and exercises codomain substitution at each application.

3.2 Problem 6: dependent identity

The dependent identity skeleton is

$$\lambda(A : \text{Type}_0).\lambda(x : A).x.$$

In the implementation, the inner domain is shifted when entering the outer binder. Applying the term twice to $x_0 : A$ yields

$$\boxed{x_0}.$$

The important property is not the trivial result but preservation of the instantiated dependent codomain: the second application is accepted only after the first codomain has been substituted and shifted correctly.

3.3 Problem 7: nested staged normalization

Consider

$$\text{unquote}()(\text{quote}()(\text{unquote}()(\text{quote}()(\text{id } x_0 \ x_0))))).$$

There are two quotation boundaries and two unquotation boundaries. The inner code normalizes to x_0 ; the outer stage then receives already-valid syntax. The observed normal form is

$$\boxed{x_0}.$$

This is a finite staged analogue of self-inspection: no quoted term executes until its corresponding unquotation.

3.4 Problem 8: open dependent application

Use the context

$$x_0 : A \rightarrow A, \quad x_1 : A.$$

The term $x_0 x_1$ is accepted and normalizes to the neutral application

$$\boxed{x_0 x_1}.$$

Because the head is unknown, NbE cannot reduce the application. It nevertheless preserves its type and returns canonical eta-long syntax when the surrounding type requires it.

4 Observed execution

The relevant executable output was:

```
problem 1: x0
problem 2: x0
problem 3: x0
problem 4: (lam (x1 x0))
problem 5: x0
problem 6: x0
problem 7: x0
problem 8: (x0 x1)
```

Before these examples, the executable also checked normalization idempotence, universe successor formation, Pi universe maxima, rejection of an ill-typed application, typed quotation, staged self-application, closure reification, and the substitution invariant.

5 What the experiments establish

The four elementary cases demonstrate beta contraction, inert quotation, stage execution, and eta expansion. The four medium cases demonstrate that the same mechanisms compose: higher-order closures can be applied repeatedly; dependent codomains are instantiated by capture-avoiding substitution; nested staging preserves type boundaries; and open applications remain neutral rather than being incorrectly reduced.

These are experiments, not a proof that every extension is sound. Natural numbers, inductive families, equality eliminators, and user recursion would require additional syntax, typing rules, and metatheoretic proofs. Within the implemented strongly normalizing fragment, however, the results provide executable evidence for the central NbE properties.